S8 Soft Processor for Spartan-6 LX9

Complete Technical Specification and Implementation Plan (v1.1 — Farad-1 board verified)

S8 Project

October 2026

Document Control
- How to read this document
- Invariants (never violate these)
- Change log versus the original manual

1. Introduction and scope
- 1.1 What S8 is
- 1.2 What this document guarantees, and what it does not
- 1.3 Non-goals (v1)

2. System architecture

3. Project directory structure

4. Top-level block diagram (textual)

5. Board fact sheet — Farad-1 (VERIFIED in v1.1)
- 5.1 Verified facts (formerly open items B1–B9)
- 5.2 Farad-1 pin table (S8-relevant)
- 5.3 Remaining hardware checks (bring-up time; not design unknowns)
- 5.4 Spartan-6 LX9 resource budget (XC6SLX9-2TQG144C, DS160 family table)

6. Instruction set architecture v1 (FROZEN)
- 6.1 Programmer's model
- 6.2 Instruction formats
- 6.3 Opcode map
- 6.4 Instruction reference
- 6.5 Flag formulas (normative)
- 6.6 HALT, TRAP, and stack faults
- 6.7 Golden encoding vectors
- 6.8 Assembler syntax
- 6.9 Worked programs (normative listings)

7. Memory architecture
- 7.1 Program memory
- 7.2 Data memory
- 7.3 MMIO register map (ports 0–7 at addresses 0xF0–0xF7)

8. CPU microarchitecture

- 8.1 Datapath
- 8.2 Control FSM (multicycle, one instruction per state sequence)
- 8.3 Bus interface and wait states
- 8.4 Clock and reset

9. RTL module specifications
10. UART subsystem
    - 10.1 Baud generation
    - 10.2 Framing
    - 10.3 FIFOs and errors
11. Mode selection and boot sequence
    - 11.1 MODE pin semantics
    - 11.2 Boot flow
    - 11.3 Copy details
12. Packet protocol v1 (byte-exact)
    - 12.1 Frame layout
    - 12.2 CRC16
    - 12.3 Commands
    - 12.4 Responses
    - 12.5 Sequencing, duplicates, timeouts
13. W25Q64 SPI flash subsystem
    - 13.1 Commands used
    - 13.2 Byte engine vs command controller
    - 13.3 Controller FSM
    - 13.4 Storage layout (Farad-1: SHARED configuration flash — relocated base)
    - 13.5 Metadata record (32 bytes, at offset 0 of a metadata sector)
    - 13.6 Atomic update procedure (device side, on COMMIT)
    - 13.7 Boot-time validation (recap of the algorithm)
14. Rust host toolchain
    - 14.1 Workspace
    - 14.2 `s8-isa`
    - 14.3 `s8-assembler`
    - 14.4 `s8-image`
    - 14.5 `s8-serial`
    - 14.6 `s8-disasm`
    - 14.7 `s8-cli`
15. Xilinx ISE workflow
16. Verification plan
    - 16.1 RTL testbenches
    - 16.2 Behavioral W25Q64 model (`tb_w25q64_model.v`)
    - 16.3 Host-side tests
    - 16.4 Hardware acceptance tests (per phase)
    - 16.5 Budgets and traceability
17. Safety, reliability, and failure handling
18. Phased execution plan
19. Future work (specified, not scheduled)
    - 19.1 Compiler (Phase 7)
    - 19.2 Instruction-fetch from external flash
    - 19.3 ABI proposal (freeze when the compiler starts)
20. References

Appendix A — Guide for an implementing agent

Document Control

Field	Value
Title	S8 Soft Processor for Spartan-6 LX9 — Complete Technical Specification and Implementation Plan
Version	1.1 (supersedes v1.0 and the original *S8 Technical Implementation Manual*)
Status	Implementation-ready specification. Design intent is frozen unless a change control entry says otherwise.
Target FPGA	XC6SLX9-2TQG144C on the **Farad-1** board — verified (schematic renders + user manual + vendor UCF)
Target flash	W25Q64BV 64-Mbit SPI NOR — **shared FPGA configuration flash** (verified; S8 storage relocated, Section 13.4)
HDL	Verilog-2001, Xilinx ISE 14.7 flow
Host tools	Rust (stable), Cargo workspace
Audience	Any engineer or coding agent implementing the RTL, the host toolchain, or the verification environment from scratch

How to read this document

This specification is written so that an implementer with no prior context can build the complete S8 system — RTL, host toolchain, and verification — without consulting the original manual. The original manual is retained as background reading only; **where the two documents disagree, this specification wins**.

Reading order for a new implementer: Section 1 (scope) → Section 6 (ISA v1, frozen) → Section 7 (memory map) → Section 8 (CPU microarchitecture) → Section 9 (RTL modules) → Sections 10–13 (peripherals and boot) → Section 14 (Rust toolchain) → Section 16 (verification) → Section 18 (phase plan).

Invariants (never violate these)

1. **The `s8-isa` Rust crate and the encoding tables in Section 6 are the single source of truth for instruction encodings.** RTL localparams and assembler opcodes must both trace to those tables. If they ever disagree, one of them is wrong — fix it against Section 6, do not "average" the two.
2. **Every golden vector in Section 6.7 must be a regression test in both the Rust workspace and the RTL testbenches.** The three starter-continuity words (`0x1001`, `0x5000`, `0x4002`) must pass in every environment.
3. **Never guess board facts.** The Farad-1 fact sheet (Section 5) is verified against the schematic renders and is the only source of board truth; anything not listed there stays TODO until verified physically. The three remaining hardware-confirmation checks are listed in Section 5.3.
4. **All lengths, addresses, loop counts, and waits are bounded.** Every FSM has a timeout; every host loop has a retry cap; every image is range-checked before use.
5. **Never erase or program the sector holding the FPGA configuration image** if the flash turns out to be shared (Section 13.4).

6. **The active application slot is never erased while a replacement has not been fully committed.** Commit order: erase → program payload → read back → program metadata marker last.
7. **CRC16 is integrity checking, not security.** If adversarial uploads ever matter, add authentication; do not pretend CRC is a signature.

Change log versus the original manual

The original manual was a sound plan with several internal inconsistencies. This version resolves all of them; the identifiers (P1–P6) refer to the analysis that found them.

ID	Problem in original manual	Resolution in this document
P1	MMIO registers at 0xF0–0xF6 fell *inside* the proposed 1 KiB data RAM (0x000–0x3FF); the "decode outside RAM" rule was unsatisfiable.	Data address space is exactly 8 bits (256 B). RAM = 0x00–0xEF, MMIO = 0xF0–0xFF. Single decode rule: `addr[7] & addr[6] & addr[5]` (addr ≥ 0xF0) selects MMIO. The published 0xF0–0xF6 register map is preserved.
P2	16 KiB flash slots could never boot: program RAM is 8 KiB (4096 × 16-bit).	Slots are 8 KiB, exactly matching program RAM capacity. The assembler and the image packer both enforce the 8192-byte limit.
P3	ISA v0 had no `IN`, no conditional branch, no `ANDI`, no peripheral-select field, and LDI could load only 0–63 — the manual's own UART-echo example could not be assembled.	ISA v1 (Section 6) is frozen with `IN`, seven conditional branches, 8-bit LDI immediate, ALU-immediate group, peripheral port fields in `IN`/`OUT`, stack ops, and shifts. Every example program in this document assembles with v1 encodings, and the three useful starter vectors are preserved bit-for-bit.
P4	CRC16 parameters and the ACK frame were unspecified.	CRC16-CCITT-FALSE is fully pinned (poly 0x1021, init 0xFFFF, no reflection, no final XOR, MSB-first) with the canonical check value 0x29B1 for ASCII "123456789". The ACK frame layout and all eight status codes are byte-exact (Section 12).
P5	The bootloader's `RUN` command conflicted with the latched MODE semantics.	`RUN` performs a warm hand-off: CPU reset is released, the bootloader FSM parks in a terminal RUN state, and re-entering programming mode requires a hard reset with MODE=1 (Section 11.1).
P6	GPIO width unstated; ISE 14.7 practicalities missing; several tables misaligned.	GPIO is exactly 4 bits, stated everywhere. Section 15 documents the ISE 14.7 environment (32-bit installer, VM guidance). All tables in this document were proofed in rendered form.

ID	Problem in original manual	Resolution in this document
—	Metadata records for atomic update were both proposed in one 4 KiB sector, so erasing to update one record destroyed the other's redundancy.	Two metadata records live in two separate 4 KiB sectors; a commit erases only the inactive record's sector (Section 13.5).
B-facts	v1.0 left all board facts (B1–B9) open and assumed a logic-level USB-UART bridge and a 25 MHz default clock.	v1.1 verifies every fact against the Farad-1 schematic (page renders inspected visually): 50 MHz clock (p85), active-low debounced reset button (p87), MODE = DIP0 (p84, inverted), host link = FT2232H Channel B UART (p83/p82, one-time FT_PROG) with DB9/MAX232 fallback (p41/p40), and the W25Q64BV confirmed as the **shared configuration flash** — storage layout relocated to base 0x060000 (Section 13.4) and the full flash pin map pinned in the populated UCF (Section 15), including the no-pull-up WP#/HOLD# hazard (p62/p61 must be driven high).

1. Introduction and scope

1.1 What S8 is

S8 is a small, custom 8-bit soft processor for learning and experimentation on a Spartan-6 LX9 FPGA. The FPGA holds the CPU, instruction and data storage, GPIO, UART, SPI flash controller, and boot control. A PC-side Rust toolchain assembles source code into an image and transfers it to the board over USB-UART. A W25Q64-family SPI NOR flash stores application images persistently so that new programs can be installed without rebuilding or re-transmitting the FPGA bitstream.

1.2 What this document guarantees, and what it does not

This document specifies the complete system at implementation level: encodings, memory maps, packet bytes, metadata records, module interfaces, FSM state lists, test inventories, and phase gates. Implementing it produces a working system without further design decisions, except the board facts listed in Section 5.

This document does **not** claim the system currently exists. As of v1.0, nothing beyond a Phase-0 learning skeleton (demo CPU with a hard-coded program, placeholder pins, a divided demo clock) has ever been synthesized. Sections 10–14 specify things that must be built and verified.

1.3 Non-goals (v1)

- No interrupts. All peripheral interaction is polling through status bits.
- No external memory bus. Program storage is internal BRAM only.

- No multiply/divide instructions, no barrel-shift-by-immediate encoding (shifts are by register value).
- No caches, no pipelines (the CPU is a multicycle FSM).
- No cryptographic authentication of uploads.
- No C-like compiler until Phase 7; the v1 toolchain is assembler-level only.

---

2. System architecture

Layer	Responsibilities
PC / host	Rust CLI: assembler, image packer, serial uploader, disassembler, serial monitor
Host link	FT2232H: Channel A = USB JTAG (iMPACT); Channel B = USB-UART to S8 (p83/p82, after one-time FT_PROG); fallback DB9 RS-232 via MAX232 (p41/p40)
FPGA top	Clock/reset conditioning, synchronized MODE, CPU, memories, UART, GPIO, SPI, boot control
S8 CPU	Fetch → decode → execute; 8 × 8-bit registers; ALU; flags; 12-bit PC; 8-bit stack pointer
Peripherals	UART RX/TX with FIFOs, GPIO ports, SPI byte engine, W25Q64 command controller
W25Q64	Nonvolatile metadata records, application slots A/B, image CRC

Clocking policy. One primary clock domain for the entire design (`clk` = board oscillator). Baud generation, CPU slowdowns (if ever needed), and all timing use **clock-enable pulses**, never fabric-generated derived clocks. If a future requirement forces a second clock domain, explicit two-flop synchronizers and timing constraints are mandatory; no such requirement exists in v1.

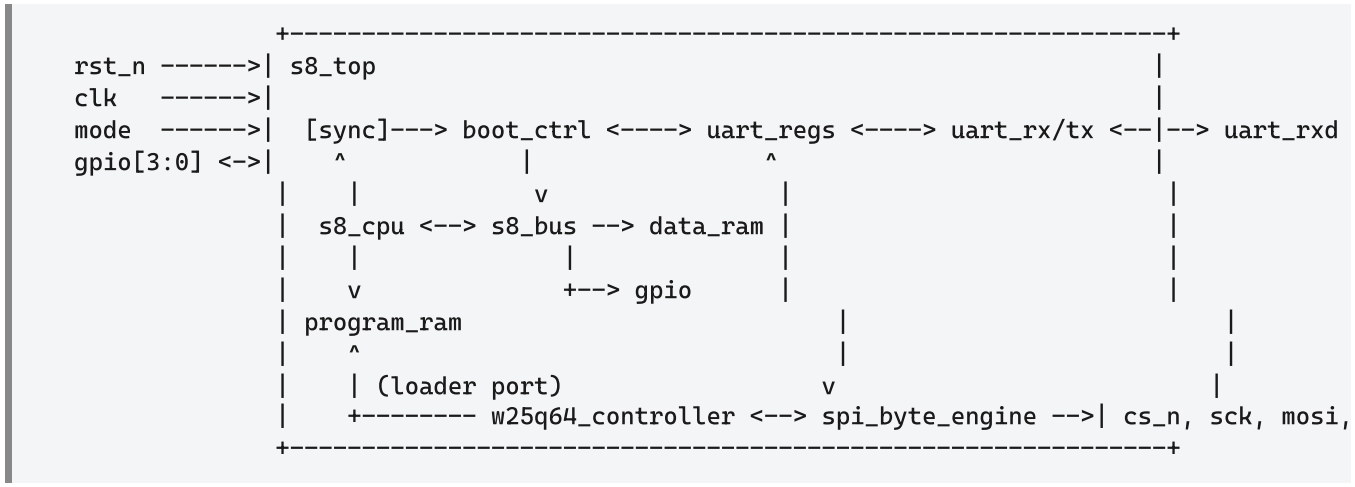
Reset policy. One external active-low reset input `rst_n`, asserted asynchronously, deasserted synchronously through a two-flop reset synchronizer. Every sequential element uses the synchronized reset.

---

3. Project directory structure

```
s8-project/
  README.md
  docs/
    isa-v1.md                 # frozen copy of Section 6 (single source of truth)
    boot-protocol.md          # frozen copy of Section 12
    flash-layout.md           # frozen copy of Section 13.4-13.5
    board-bringup.md          # verified board facts as they are discovered
    verification-plan.md      # frozen copy of Section 16
  rtl/
    s8_top.v
    s8_cpu.v
    s8_alu.v
    s8_regfile.v
    s8_program_ram.v
    s8_data_ram.v
    s8_bus.v
    s8_gpio.v
    uart_rx.v
    uart_tx.v
    uart_regs.v
    spi_byte_engine.v
    w25q64_controller.v
    s8_boot_ctrl.v            # MODE handling + bootloader FSM + packet parser
    baud_gen.v
  tb/
    tb_s8_alu.v
    tb_s8_regfile.v
    tb_s8_cpu_programs.v
    tb_uart_loopback.v
    tb_spi_byte_engine.v
    tb_w25q64_model.v         # behavioral flash model used by other testbenches
    tb_boot_packets.v
    tb_full_boot.v
    vectors/                  # golden vectors exported from s8-isa (generated)
  constraints/
    board.ucf                 # populated only from verified schematic facts
  host/
    s8/                       # Rust workspace
      Cargo.toml
      crates/
        s8-isa/
        s8-assembler/
        s8-image/
        s8-serial/
        s8-disasm/
        s8-cli/
  examples/
    gpio.s8
    uart_echo.s8
    stack_demo.s8
    flash_info.s8
  tools/
    gen_isa_vectors.js        # emits golden vectors for both RTL and Rust tests
  build/                      # generated artifacts; bitstreams not committed by default
```

4. Top-level block diagram (textual)



Ownership rule: while MODE is latched as programming mode, the CPU is held in reset and the boot controller is the only master of program RAM, the SPI controller, and the UART data path. While running an application, the CPU owns program RAM, data RAM, and the UART/GPIO/SPI-register-visible peripherals; the flash controller is idle unless the application drives it through a future extension (not in v1: SPI registers are not CPU-visible in v1; flash access belongs to the boot controller only).

---

5. Board fact sheet — Farad-1 (VERIFIED in v1.1)

Target board: **Farad-1** (Farad Andish, schematic rev 2016-12-29). In-repo evidence: `BoardFiles/Farad-1-Schematic.pdf` (6 sheets), `BoardFiles/Farad-1-User-Manual.pdf`, `BoardFiles/Farad1_UCF.ucf`. Every fact below was verified by inspecting **page renders of the schematic** (not text dumps — net-label/pin associations were confirmed visually), cross-checked against the vendor UCF and manual. This table is the single source of board truth for S8.

5.1 Verified facts (formerly open items B1–B9)

#	Fact	Verified value	Evidence
B1	FPGA part	**XC6SLX9-2TQG144C** → ISE part string `xc6slx9-2tqg144` (speed grade −2, TQG144, commercial)	Schematic title blocks (all 6 sheets); user manual
B2	System clock	**50.000 MHz** oscillator (Y2) → **p85** (net OSC, bank 1). PERIOD = 20.000 ns. External SMA clock on p88 exists but is unused.	Peripherals sheet (Y2); sheet 2 U1B; vendor UCF `Clock = p85`

#	Fact	Verified value	Evidence
B3	Reset	Button **S2** → **nFP_RST, p87**, **active low**, 4.7 K pull-up (R22) + 4.7 µF (C46) — hardware-debounced. Maps directly to `rst_n`. S1 is PROGRAM_B; do not use it as a user reset.	Peripherals sheet; sheet 2 U1B (p87)
B4	Host link	**No logic-level USB-UART bridge exists.** Two safe paths: **(a) FT2232H Channel B as UART** — BDBUS0 (TXD from FTDI) → net FT_D0 → **p83** (FPGA in); BDBUS1 (RXD to FTDI) ← net FT_D1 ← **p82** (FPGA out); 3.3 V CMOS; requires a one-time FT_PROG EEPROM change (Channel B = UART, no flow control; Channel A stays JTAG). **(b) DB9 + MAX232** — FP_TX **p41** → T1IN → DB9 pin 2 (straight-through cable); DB9 pin 3 → R1IN → R1OUT → divider R29 5.6 K / R30 10 K → **p40** (≈3.2 V max — safe for the 3.3 V bank, input-only). Path (a) is the S8 default.	Peripherals sheet (MAX232 + divider inspected); sheet 2 U1C (FP_TX/FP_RX on p41/p40 — the vendor UCF's `Serial_Tx = p41`, `Serial_Rx = p40` is correct)
B5	GPIO (v1)	**LED0–LED3 = p105, p104, p102, p101**, **active HIGH** (390 Ω → LED → GND). Extras (unconstrained in v1): 7-seg DISP0–7 = p100, 99, 98, 97, 95, 94, 93, 92 (active HIGH, common cathode); DIP1–3 = p81, p79, p74 (pull-ups, 0 = down); DS18B20 = p35; EXT_CLK = p88.	Peripherals sheet (LED/DISP/DIP polarity inspected); sheet 2 U1B/U1D; vendor UCF
B6	MODE strap	**No dedicated MODE pin.** Use **DIP0 = p84** (4.7 K pull-up, switch to GND: open = 1, closed = 0). Farad-1 adaptation: `MODE_INVERT = 1` in `s8_top`, so **switch open (all-off default) = run application**, switch closed = programming. Spec semantics (logical 1 = programming) are unchanged.	Peripherals sheet (pull-ups); sheet 2 U1B (p84)

#	Fact	Verified value	Evidence
B7	Flash sharing	**SHARED: the W25Q64BV is the FPGA configuration flash**, master SPI. MSEL straps: M0 tied to VCCO (1) at p69, M1 tied to GND (0) at p60, M2 unconnected (internal pull-down = 0). The bitstream occupies flash 0x000000 upward (~338 KB for the LX9). S8 storage is relocated (Section 13.4); the configuration region is untouchable.	User manual (.mcs flash-programming flow); sheet 2 U1C (straps inspected); sheet "System Memory" (U8)
B8	Flash part and pins	**W25Q64BV**, JEDEC ID expected **0xEF 0x40 0x17**. Wiring: **SCK = p70** (CCLK, net FP_PR_CLK, board pull-up R23 4.7 K) → CK(6); **CS# = p38** (CSO_B, net FP_CSO, board pull-up R6 4.7 K) → S(1); **MOSI = p64** (net MISO0) → DI(5); **MISO = p65** (net MISO1) ← DO(2); **WP# = p62** (net MISO2) → pin 3 — **no board pull-up: S8 must drive HIGH**; **HOLD# = p61** (net MISO3) → pin 7 — **no board pull-up: S8 must drive HIGH**.	System Memory sheet (U8 + pull-up placement inspected); sheet 2 U1C (pin↔net inspected)
B9	I/O voltages	**All four banks VCCO = +3.3 V** (LM1117-3.3). Every S8 port: **LVCMOS33**.	FPGA Power + Power Regulator sheets

5.2 Farad-1 pin table (S8-relevant)

Function	Pin	Dir	Notes
clk	p85	in	50 MHz; PERIOD 20.000 ns
rst_n	p87	in	Active low, RC-debounced button
mode	p84	in	DIP0; inverted in RTL (open = run app)
gpio[3:0]	p105, p104, p102, p101	out	LEDs, active high
uart_rxd	p83	in	FTDI Ch.B TXD (path a)
uart_txd	p82	out	FTDI Ch.B RXD (path a)
uart2_rxd	p40	in	DB9 via MAX232 + 5K6/10K divider (path b)
uart2_txd	p41	out	DB9 via MAX232
spi_sck	p70	out	Flash CK (CCLK pin, net FP_PR_CLK)
spi_cs_n	p38	out	Flash CS# (CSO_B; board pull-up R6)

Function	Pin	Dir	Notes
spi_mosi	p64	out	Flash DI (net MISO0)
spi_miso	p65	in	Flash DO (net MISO1)
spi_wp_n	p62	out	Drive HIGH — no board pull-up
spi_hold_n	p61	out	Drive HIGH — no board pull-up

Left unconstrained and unused in v1: SRAM IS61WV5128 (512 KiB × 8: MEM_D0-7 = p142, 141, 140, 139, 112, 111, 119, 120; ADD0-18 = p6, 5, 2, 1, 144, 137, 134, 133, 132, 131, 118, 117, 116, 115, 114, 123, 124, 126, 127; MEM_CE = p143, MEM_WE = p138, MEM_OE = p121 — reserved for a future XIP/data-expansion phase), 7-segment, DIP1–3, DS18B20, EXT_CLK, FTDI FIFO handshake lines.

5.3 Remaining hardware checks (bring-up time; not design unknowns)

1. **Flash-pin confirmation**: the B8 map was read from schematic renders; confirm it on hardware with the Phase-5 JEDEC read. If 0x9F fails, swap p64/p65 (MOSI/MISO) first; a power-off continuity check (flash pin ↔ FPGA pin, diode mode) settles it in minutes.
2. **FT_PROG step**: back up the FT2232H EEPROM, then set Channel B function = UART (no flow control) once. Record the backup in `docs/board-bringup.md`.
3. **VCCAUX**: vendor power design (2.5 V per UG380 expectations); S8 does not use it — recorded so nobody wires analog or aux-dependent logic into the plan.

5.4 Spartan-6 LX9 resource budget (XC6SLX9-2TQG144C, DS160 family table)

Resource	Device total	S8 v1 use	Remaining
Logic cells	9,152	est. 3,000–4,500 (CPU + UART + SPI + boot control)	ample
Slices (4 LUT + 8 FF each)	1,430	est. 700–900 ≈ **50–60 %** (budget gate: ≤ 60 %, Section 16.5)	~500+ for compiler/debugger phases
Block RAM (18 Kb blocks)	32 (576 Kb total)	**4** — program RAM (4096 × 16 = 64 Kb in 2048×9 mode, 2 wide × 2 deep)	28
Distributed RAM (max)	90 Kb	small — data RAM (240 × 8) + 2 × 16-byte UART FIFOs (~32 LUTs)	ample
DSP48A1 slices	16	**0** (no multiply instruction in ISA v1)	all 16
Global clock nets used	4 × 16 (4 clock regions)	**1** (oscillator, p85); no CMT/DCM — baud via phase accumulator	—
CMTs (DCM + PLL)	2	0	—
User I/O (TQG144 package)	102	~12 S8 ports (clk, rst, mode, 4 LED, 2 UART, 4–6 SPI) + SRAM/7-seg/DIP reserved	rest unused in v1

Notes: the design fits comfortably; the tightest axis is **slices** (not BRAM). If utilization approaches the 60 % gate, the first reductions are: RX FIFO depth 16 → 8, optional GPIO_DIR register removal, and the ROL/ROR shift variants (funct space already reserved). BRAM usage is deliberately conservative so a future XIP/instruction-fetch phase (Section 19.2) can claim 4–8 more blocks. Sources: DS160 family overview table (logic cells / slices / FFs / distributed RAM / DSP / BRAM per device) — cross-checked against two independent part summaries.

---

6. Instruction set architecture v1 (FROZEN)

Version note: v1 replaces the unfrozen starter encoding. Three starter instructions keep their exact words (`LDI R0,1` = `0x1001`, `OUT GPIO_OUT,R0` = `0x5000`, `JMP 0x002` = `0x4002`) so starter testbenches still carry value. Any future change requires ISA v2, a new document revision, and regeneration of all golden vectors.

6.1 Programmer's model

Resource	Size	Notes
R0–R7	8 × 8-bit	General purpose. R0 is **not** hardwired to zero. No register is reserved by hardware; the ABI (Section 19.3) reserves some by convention.
PC	12-bit	Word address, range 0x000–0xFFF. Instruction words are 16 bits.
SP	8-bit	Stack pointer, byte address in data space. Reset value 0xF0 (empty stack).
Flags	Z, C, N, O	Zero, Carry, Negative (bit 7), oVerflow (signed). Updated only by instructions listed in 6.5.
Halt bit	1	Set by `HALT`; cleared by reset or by writing CPU_CONTROL.resume.
Trap bit	1	Set by illegal opcode / illegal branch condition / stack fault; cleared only by reset.

6.2 Instruction formats

All instructions are exactly 16 bits. `op` = bits [15:12].

Format	Fields	Used by
R (register)	op[15:12], rd[11:9], rs[8:6], funct[5:3], 000[2:0]	ALU reg-reg, SHIFT
I8 (immediate 8)	op[15:12], rd[11:9], imm8[7:0]	LDI
RI (reg + imm6)	op[15:12], rd[11:9], funct[8:7], 0[6], imm6[5:0]	ADDI/ANDI/ORI/CMPI
M (memory)	op[15:12], rd[11:9], rs[8:6]	LD, ST

Format	Fields	Used by
A (address)	op[15:12], addr12[11:0]	JMP, CALL
P (port)	OUT: op, port[11:9], rs[8:6]. IN: op, rd[11:9], port[8:6]	IN, OUT
B (branch)	op[15:12], cond[11:9], rel8[7:0]	Jcc
S (single)	op[15:12], 000000000000	RET, HALT
U (unary reg)	op[15:12], rd[11:9], 000000000	PUSH, POP

Unused bits must be encoded as zero. Assemblers must emit zero for them; decoders must treat non-zero unused bits as an **illegal opcode** (trap), so that encoding drift is detected early rather than silently tolerated.

6.3 Opcode map

op	Mnemonic	Format	Semantics summary
0x0	ALU reg-reg	R	ADD, SUB, AND, OR, XOR, CMP, MOV, NOT (funct 0–7)
0x1	LDI	I8	rd ← zero-extend(imm8)
0x2	ALU imm	RI	ADDI, ANDI, ORI, CMPI (funct 0–3), imm6
0x3	LD	M	rd ← M[rs]
0x4	JMP	A	PC ← addr12
0x5	OUT	P	Port[port] ← Rs
0x6	Jcc	B	Conditional PC ← PC + 1 + sext(rel8)
0x7	CALL	A	Push return address; PC ← addr12
0x8	ST	M	M[rs] ← rd
0x9	RET	S	PC ← pop
0xA	IN	P	rd ← Port[port]
0xB	PUSH	U	Push rd
0xC	POP	U	Pop into rd
0xD	SHIFT	R	SHL, SHR, ROL, ROR (funct 0–3), amount = Rs[2:0]
0xE	reserved	—	**Illegal opcode → TRAP**
0xF	HALT	S	Halt until reset or resume

6.4 Instruction reference

Field names below refer to Section 6.2. `M[x]` is the data-space byte at address x. `sext` is sign extension to 8 bits. `PC_after` = address of the instruction following the current one (PC + 1, wrapping within 12 bits).

Mnemonic	Encoding	Semantics	Flags
ADD rd, rs	0x0, funct=0	{C, rd} ← rd + rs (9-bit sum)	Z C N O
SUB rd, rs	0x0, funct=1	{B, rd} ← rd − rs; C = NOT borrow (C=1 iff rd ≥ rs)	Z C N O
AND rd, rs	0x0, funct=2	rd ← rd & rs	Z, N; C←0, O←0
OR rd, rs	0x0, funct=3	rd ← rd | rs	Z, N; C←0, O←0
XOR rd, rs	0x0, funct=4	rd ← rd ^ rs	Z, N; C←0, O←0
CMP rd, rs	0x0, funct=5	Compute rd − rs; discard result, keep flags only	Z C N O
MOV rd, rs	0x0, funct=6	rd ← rs	Z, N; C←0, O←0
NOT rd, rs	0x0, funct=7	rd ← ~rs (note: source is rs, not rd)	Z, N; C←0, O←0
LDI rd, imm8	0x1	rd ← imm8	Z, N (of result); C←0, O←0
ADDI rd, imm6	0x2, funct=0	rd ← rd + zext(imm6)	Z C N O
ANDI rd, imm6	0x2, funct=1	rd ← rd & zext(imm6)	Z, N; C←0, O←0
ORI rd, imm6	0x2, funct=2	rd ← rd | zext(imm6)	Z, N; C←0, O←0
CMPI rd, imm6	0x2, funct=3	flags only, from rd − zext(imm6)	Z C N O
LD rd, [rs]	0x3	rd ← M[rs]	unchanged
ST rd, [rs]	0x8	M[rs] ← rd (base in rs, data in rd — mirrors LD)	unchanged
JMP addr12	0x4	PC ← addr12	unchanged
JZ / JNZ / JC / JNC / JN / JP / JO rel8	0x6, cond 0–6	if condition: PC ← PC_after + sext(rel8)	unchanged
(cond=7)	0x6	**Illegal → TRAP**	unchanged
CALL addr12	0x7	push PC_after; PC ← addr12	unchanged
RET	0x9	PC ← pop	unchanged
IN rd, port	0xA	rd ← Port[port]	unchanged
OUT port, rs	0x5	Port[port] ← rs	unchanged
PUSH rd	0xB	push rd	unchanged
POP rd	0xC	rd ← pop	unchanged
SHL rd, rs	0xD, funct=0	{C, rd} ← rd << Rs[2:0] (C = last bit out)	Z N C; O←0
SHR rd, rs	0xD, funct=1	logical right shift; C = last bit out	Z N C; O←0
ROL rd, rs	0xD, funct=2	rotate left through rd, no C participation	Z N; C←0, O←0
ROR rd, rs	0xD, funct=3	rotate right through rd, no C participation	Z N; C←0, O←0

Mnemonic	Encoding	Semantics	Flags
HALT	0xF	Halt bit ← 1; CPU stops retiring until resume or reset	unchanged

Semantics notes.

- Branch targets: `target = (PC_after + sext(rel8)) mod 4096`. The assembler computes `rel8 = target_label − PC_after`. Range −128..+127 from PC_after; out-of-range is an assembler error. Automatic relaxation (inverted branch over a `JMP`) is specified as assembler behavior for v1: when a `Jcc` target is out of range, the assembler emits `inverted-cc +3; JMP target` and reports the rewrite in verbose mode.
- `LD/ST` addresses are full 8-bit data-space addresses held in rs; the MMIO region 0xF0–0xFF is reachable by ordinary `LD/ST` but the canonical access path is `IN/OUT` (identical hardware decode; `IN/OUT` exist for code density and clarity).
- Shift amounts use only rs[2:0]; a shift by 8 behaves as a shift by 0 (copies rd to rd, sets Z/N from the unchanged value, C←0 for SHL/SHR).

6.5 Flag formulas (normative)

Let a, b be the 8-bit operands, and r the 8-bit result.

- ADD/ADDI: `sum9 = {1'b0,a} + {1'b0,b}`; `r = sum9[7:0]`; `C = sum9[8]`; `Z = (r == 0)`; `N = r[7]`; `O = (a[7] == b[7]) && (r[7] != a[7])`.
- SUB/CMP/CMPI: `diff9 = {1'b0,a} − {1'b0,b}`; `r = diff9[7:0]`; `C = ~diff9[8]` (C = 1 when a ≥ b, unsigned); `Z, N` from r; `O = (a[7] != b[7]) && (r[7] != a[7])`.
- AND/OR/XOR/NOT/MOV/LDI: `Z, N` from result; `C = 0`; `O = 0`.
- SHL: `C` = last bit shifted out (the bit leaving position 7); SHR likewise from position 0; ROL/ROR: `C = 0`.
- All other instructions leave every flag unchanged.
- Reset clears Z, C, N, O.

6.6 HALT, TRAP, and stack faults

- `HALT` sets the halt bit. The CPU remains in the HALT state, keeps servicing nothing, and exits only via CPU_CONTROL.resume (port 6, bit 0) or reset. Halt is a *legitimate* program state: BOOT_STATUS and CPU_CONTROL remain readable while halted.
- TRAP is entered on: opcode 0xE; any R-format instruction with non-zero bits [2:0]; RI-format with bit 6 set; any unused field pattern not defined above; branch condition 7; PUSH/POP/CALL/RET when SP is outside the legal range 0xE0–0xF0 before or after the operation. TRAP latches the trap bit; the CPU stops retiring until reset. Trap is not resumable by design: it indicates a bug, and the debug strategy is reset + BOOT_STATUS/CPU_CONTROL inspection.
- Stack bounds: SP starts at 0xF0 (empty). PUSH/CALL decrement before use; POP/RET read then increment. Usable depth is 16 entries (0xEF down to 0xE0). A faulting stack operation leaves SP and memory unmodified (check before commit).

6.7 Golden encoding vectors

These words are normative. They were machine-generated from the encoding rules above and must appear verbatim as pass criteria in the Rust tests (`s8-isa`) and the RTL instruction-vector testbench.

Assembly	Word
`LDI R0, 1`	`0x1001`
`OUT GPIO_OUT, R0`	`0x5000`
`JMP 0x002`	`0x4002`
`ADD R2, R3`	`0x04C0`
`SUB R7, R1`	`0x0E48`
`AND R4, R5`	`0x0950`
`OR R0, R7`	`0x01D8`
`XOR R6, R6`	`0x0DA0`
`CMP R1, R2`	`0x02A8`
`MOV R1, R0`	`0x0230`
`NOT R4`	`0x0838`
`NOP` (= `MOV R0, R0`)	`0x0030`
`LDI R7, 255`	`0x1EFF`
`LDI R3, 0xA5`	`0x16A5`
`ADDI R0, 5`	`0x2005`
`ANDI R1, 0x0F`	`0x228F`
`ORI R2, 0x30`	`0x2530`
`CMPI R1, 0`	`0x2380`
`LD R1, [R0]`	`0x3200`
`LD R5, [R7]`	`0x3BC0`
`ST R1, [R0]`	`0x8200`
`ST R0, [R7]`	`0x81C0`
`JMP 0xFFF`	`0x4FFF`
`CALL 0x123`	`0x7123`
`RET`	`0x9000`
`JZ .-3`	`0x60FD`
`JNZ .-1`	`0x62FF`
`JC .+127`	`0x647F`
`JN .-128`	`0x6880`
`IN R1, UART_STATUS`	`0xA2C0`
`IN R0, UART_DATA`	`0xA100`

Assembly	Word
`IN R2, GPIO_IN`	`0xA440`
`OUT UART_DATA, R0`	`0x5800`
`OUT GPIO_DIR, R3`	`0x54C0`
`PUSH R5`	`0xBA00`
`POP R5`	`0xCA00`
`SHL R2, R3`	`0xD4C0`
`SHR R2, R3`	`0xD4C8`
`ROL R1, R4`	`0xD310`
`ROR R1, R4`	`0xD318`
`HALT`	`0xF000`

Note: instruction word `0x0000` is `ADD R0, R0`, a **real instruction**, not a NOP. Programs must never rely on zero-filled memory executing as NOP. The assembler pseudo-op `NOP` emits `0x0030` (`MOV R0, R0`).

The generator that produced this table (`tools/gen_isa_vectors.js`) also exports machine-readable vectors to `tb/vectors/` for the RTL testbenches and to the Rust test data; regenerate after any approved ISA change.

6.8 Assembler syntax

- Case-insensitive mnemonics and register names (`R0`–`R7`).
- Comments start with `;` and run to end of line.
- Labels: `name:` at the start of a line, bound to the next emitted word address.
- Numbers: decimal (`42`), hex (`0x2A`), binary (`0b101010`). Negative decimals allowed for relative branch operands.
- Register indirect operands: `[Rn]`.
- Branch operands: label or explicit relative value (`.-3`, `.+2`, or a raw number interpreted as relative offset — labels are strongly preferred).
- Port names are predefined symbols: `GPIO_OUT`=0, `GPIO_IN`=1, `GPIO_DIR`=2, `UART_STATUS`=3, `UART_DATA`=4, `BOOT_STATUS`=5, `CPU_CONTROL`=6.
- Directives: `.org <word_addr>` (set location counter for program memory), `.word <w>[, <w>...]` (emit raw 16-bit words), `.byte <b>[, <b>...]` (emit bytes, packed two per word, **high byte first**), `.equ <name>, <value>` (constant), `.ascii "text"` (emit bytes, high-byte-first packing, same rule as .byte).
- Pseudo-ops expanded by the assembler: `NOP` → `MOV R0,R0`; `BRA label` → `JMP label`; `LDI16 rd, <16-bit>` → `LDI rd, hi8; ORI rd, lo8` — **rejected** unless rd is documented as scratch by the programmer (it overwrites rd twice; emitted with a warning).
- Errors are mandatory (with file, line, column, and expected-vs-found): unknown mnemonic, bad register, immediate out of range for its format (imm8 > 255, imm6 > 63), duplicate label, undefined label, branch target out of relative range after relaxation, `.org` backwards overlap, image exceeding 8192 bytes.

6.9 Worked programs (normative listings)

6.9.1 GPIO blink-seed (3 words)

```
        LDI  R0, 1          ; 0x1001
        OUT  GPIO_OUT, R0   ; 0x5000
loop:   JMP  loop           ; 0x4002
```

Words: `0x1001 0x5000 0x4002`. This is the hardware smoke test for Phase 2.

6.9.2 UART echo (9 words)

```
start:  IN   R1, UART_STATUS  ; 0xA2C0
        ANDI R1, 0x01         ; 0x2281   ; RX_VALID
        JZ   start            ; 0x60FD
        IN   R0, UART_DATA    ; 0xA100   ; pop RX FIFO
wait:   IN   R1, UART_STATUS  ; 0xA2C0
        ANDI R1, 0x02         ; 0x2282   ; TX_READY
        JZ   wait             ; 0x60FD
        OUT  UART_DATA, R0    ; 0x5800
        JMP  start            ; 0x4000
```

Words: `0xA2C0 0x2281 0x60FD 0xA100 0xA2C0 0x2282 0x60FD 0x5800 0x4000`. Both relative branches resolve to −3.

6.9.3 Stack / CALL-RET demo

```
        LDI  R1, 5
        CALL work              ; R1 = 10 after return
        HALT
work:   PUSH R1
        ADD  R1, R1
        POP  R1
        ; ... recompute: the demo doubles R1 then restores — shown to exercise
        ; CALL/RET/PUSH/POP encodings; real code uses the ABI in Section 19.3.
        RET
```

The assembler test suite must round-trip this program and check it against hand-computed words.

---

7. Memory architecture

7.1 Program memory

- 4096 × 16-bit synchronous BRAM (8 KiB), word-addressed by the 12-bit PC.
- Single read port shared between the CPU fetch path and the boot loader. The loader has priority only while the CPU is held in reset; the mux in `s8_program_ram` must never grant both in the same cycle (assertion in simulation).
- Synchronous read: address registered at the clock edge, data valid on the following edge. The CPU FSM accounts for this latency (Section 8.3).

- Byte order within a word: the byte at even image offset is the **high** byte (bits [15:8]). A word at address k holds image bytes `2k` (high) and `2k+1` (low). All of assembler output, `.s8img` payloads, flash bytes, and loader logic obey this single rule.
- No write port is exposed to the application in v1; only the loader writes program RAM.

7.2 Data memory

- Data address space is 8 bits (256 bytes total).
- `s8_data_ram`: 240 × 8 synchronous RAM at 0x00–0xEF. Same single-cycle sync-read semantics as program RAM.
- Software convention (not enforced by hardware beyond stack bounds): 0x00–0xDF variables, 0xE0–0xEF stack region.
- 0xF0–0xFF is MMIO, decoded by `s8_bus` outside the RAM chip select. 0xF8–0xFF are reserved: reads return 0x00, writes are ignored, no trap.

7.3 MMIO register map (ports 0–7 at addresses 0xF0–0xF7)

Addr	Port	Name	Access	Definition
0xF0	0	GPIO_OUT	R/W	Bits [3:0] drive GPIO pads when the corresponding DIR bit is 1. Reset value 0x00.
0xF1	1	GPIO_IN	R	Bits [3:0]: two-flop-synchronized pad samples. Read-only.
0xF2	2	GPIO_DIR	R/W	Bit = 1 → pad is an output. Reset value 0x0F (all outputs; keeps the LED demo working out of reset).
0xF3	3	UART_STATUS	R	bit0 RX_VALID (RX FIFO non-empty); bit1 TX_READY (TX FIFO not full); bit2 RX_FRAME_ERR (sticky, cleared by reading UART_DATA); bit3 RX_OVERRUN (sticky, cleared by reading UART_DATA).
0xF4	4	UART_DATA	R/W	Read: pop one byte from RX FIFO (returns 0x00 if empty; clears sticky error bits). Write: push one byte to TX FIFO (write ignored when TX FIFO full).

Addr	Port	Name	Access	Definition
0xF5	5	BOOT_STATUS	R	bit0 MODE_LATCHED (1 = programming mode); bit1 IMAGE_VALID (a valid image was found at boot); bits[4:2] BOOT_ERR: 0 none, 1 no valid image, 2 image CRC mismatch, 3 flash ID mismatch, 4 flash timeout.
0xF6	6	CPU_CONTROL	R/W	Write: bit0 = 1 → clear halt (resume); bit1 = 1 → soft reset request (re-enters the boot flow with latched MODE). Read: bit0 HALTED, bit1 TRAPPED, bit2 RUNNING.
0xF7	7	—	—	Reserved, reads 0x00.

Reads of write-only fields and writes to read-only fields are benign (return 0 / ignored). All GPIO inputs are synchronized; no combinational path from any pad reaches CPU control logic.

---

8. CPU microarchitecture

8.1 Datapath

- 8 × 8-bit register file, two read ports, one write port, synchronous write, asynchronous read (distributed RAM; the file is small).
- ALU: combinational; computes result and the four flags per Section 6.5. Flags live in a flag register updated at the EXECUTE→next transition when the decoded instruction writes flags.
- PC: 12-bit, with increment, jump, branch, call (push PC_after), and return (pop) paths.
- SP: 8-bit with increment/decrement and bounds comparator against 0xE0–0xF0.
- Imm extraction unit: imm8, imm6 (zero-extended), addr12, rel8 (sign-extended), all from the latched instruction word.

8.2 Control FSM (multicycle, one instruction per state sequence)

State	Action	Next
RESET	Outputs held safe; PC ← 0; SP ← 0xF0; flags cleared	FETCH_REQ
FETCH_REQ	Drive program-RAM address = PC	FETCH_WAIT

State	Action	Next
FETCH_WAIT	Wait one cycle for synchronous read; latch instruction word	DECODE
DECODE	Decode opcode/funct; validate unused bits; extract immediates	EXECUTE, or TRAP on illegal encoding
EXECUTE	ALU operation; register file write; flag update; or port read via `IN`	MEM (for LD/ST/PUSH/POP/CALL/RET stack access), otherwise FETCH_REQ
MEM	Drive data bus: address, we/re; one cycle for synchronous RAM or MMIO	FETCH_REQ (writeback folded into MEM)
HALT	CPU idle; BOOT_STATUS/CPU_CONTROL readable	FETCH_REQ on resume, RESET on soft reset
TRAP	Latched stop	RESET only

Any state combined with an active reset returns to RESET. The multicycle structure means one instruction takes 3–5 cycles; there is no requirement to optimize this in v1. Correctness first: every state must be able to reach HALT or TRAP when the opcode demands it, and no state may retire two instructions.

8.3 Bus interface and wait states

The CPU presents to `s8_bus`: `addr[7:0]`, `re`, `we`, `wdata[7:0]`; the bus returns `rdata[7:0]` and `ready`. RAM targets answer `ready` in one cycle (the MEM state already waits one cycle). MMIO targets answer combinationally in the same cycle. If a future peripheral needs a wait state, it holds `ready` low and the CPU stays in MEM; v1 has no such peripheral, but the FSM must implement the wait so the interface is future-proof. Writes to RAM/MMIO complete only when `ready` is seen.

8.4 Clock and reset

Single clock `clk` (Farad-1: 50.000 MHz oscillator on p85 — `CLK_HZ` = 50000000 on hardware, any value in simulation). Reset: asynchronous assert, synchronous deassert, two-flop synchronizer, active low (p87 button is active low with hardware debounce). The demo divider from the starter design is **removed**; the CPU runs at full clock with no slow-down in v1.

---

9. RTL module specifications

Common conventions: Verilog-2001; every module has a synchronous reset unless purely combinational; parameters for anything board- or capacity-related; one module = one file; no latches (assert in synthesis review); registered outputs where a path would otherwise be long.

Module	Key ports	Specification
`s8_top`	clk, rst_n, mode, uart_rxd, uart_txd, gpio[3:0], cs_n, sck, mosi, miso	Wiring only. Instantiates and connects everything; owns the MODE synchronizer and the boot-mode latch. Zero guessed pin numbers (UCF only).

Module	Key ports	Specification
`s8_cpu`	instr[15:0], instr_addr[11:0], bus interface, halt/trap status	FSM per Section 8; exposes `halted`, `trapped` for CPU_CONTROL.
`s8_alu`	op_a, op_b[7:0], funct, carry-in context, result, flags_out	Combinational; implements Section 6.5 formulas exactly; unit-tested exhaustively per operation class against a software model.
`s8_regfile`	raddr_a, raddr_b, waddr, wdata, we, clk	8 × 8; sync write; async read; simulation-friendly: expose registers to TB via hierarchical reference, not extra ports.
`s8_program_ram`	cpu_addr, cpu_out (unused in v1), load_addr, load_data, load_we, sel_load	4096 × 16 sync BRAM; read-first; loader port muxed with CPU port; assertion: never both enables in one cycle.
`s8_data_ram`	addr[7:0], re, we, wdata, rdata	240 × 8 sync RAM, decode excludes 0xF0+ (chip-select input from bus).
`s8_bus`	cpu bus in; RAM chips selects; MMIO read mux	Address decode per Section 7.2; read mux prioritizes ready sources; never routes 0xF0+ to RAM.
`s8_gpio`	gpio_out, gpio_dir, pad_in, pad_out, pad_oe	4-bit; synchronizes pad_in (2FF); drives pads per DIR; safe default (DIR=0x0F, OUT=0x00 at reset).
`baud_gen`	clk, tick16 (16× baud tick), parameter CLK_HZ, BAUD	Phase accumulator (Section 10.1); fully parameterized.
`uart_rx`	rxd (already synchronized at top), tick16, rdata, valid, framing_err, overrun	Start-bit qualification (half-bit sample must still be low), 8 data LSB-first, 1 stop bit; framing error flag on bad stop; overrun when pushing a full FIFO (flag latched in `uart_regs`).
`uart_tx`	tdata, tvalid, tready, tick16, txd	8N1, LSB-first; ready/valid handshake; deterministic one-byte-per-10-bit-times throughput.
`uart_regs`	cpu bus side, rx/tx FIFO sides, status	16-byte RX FIFO, 16-byte TX FIFO (BRAM or distributed per utilization review); sticky error bits cleared on UART_DATA read; implements port 3 and 4 semantics exactly.
`spi_byte_engine`	start, tx_byte, rx_byte, done, sck, mosi, miso, cpol/cpha fixed 0/0	Shifts 8 bits MSB-first; SCLK = clk divided by 2×DIV (DIV default 4 → 6.25 MHz at the Farad-1's 50 MHz clk — well under the W25Q64BV's 80 MHz ceiling); `done` one cycle; CS owned by the controller, not here.

Module	Key ports	Specification
`w25q64_controller`	op, slot_addr[23:0], len[9:0], data FIFO side, busy, done, error[2:0]	FSM: IDLE → WREN (for writes) → COMMAND+ADDRESS → DATA or POLL_BUSY → STATUS_CHECK/TIMEOUT → DONE/ERROR. Owns CS across the full command. Busy-poll timeout 1 s of clk cycles. Page program split at 256 bytes happens *above* this module (loader) — this module refuses len > 256 for program and refuses any command whose len would cross a page (assertion + ERROR).
`s8_boot_ctrl`	mode_sync, uart FIFO side, spi controller side, program RAM loader port, cpu_reset_hold, boot_status	Top-level boot FSM (Section 11.2), packet parser (Section 12), image validation and copy, metadata commit. The largest control module; keep the packet parser as a clearly separated always block or submodule.

---

10. UART subsystem

10.1 Baud generation

16× oversampling with a phase accumulator:

```
every clk:  acc <= acc + 16*BAUD
            if acc >= CLK_HZ: acc <= acc - CLK_HZ; tick16 <= 1
            else tick16 <= 0
```

On the Farad-1 (`CLK_HZ` = 50,000,000, `BAUD` = 115200): addend = 16 × 115200 = 1,843,200; `tick16` fires on average every 27.12 clk cycles (the naive integer view — 434.028 clocks per bit — never enters the design). The long-term average is exactly BAUD, with bounded instantaneous jitter of one clk period. `CLK_HZ` stays a parameter so simulation can use any clock. RX and TX share one `baud_gen` instance only if both use the same phase; simpler and preferred: one instance, one `tick16` wire, both consume it.

10.2 Framing

8 data bits, no parity, 1 stop bit; LSB first on the wire. RX: two-flop synchronize `uart_rxd` at the top level (before any edge logic); on falling edge, wait 8 ticks (half bit), confirm start still low (else return to idle — glitch rejection); sample each data bit at mid-bit (16 ticks apart); check stop bit at mid-stop; framing error flag if stop ≠ 1 (byte still delivered, flagged). TX: standard serializer; `tready` deasserts while the shift register is busy and the one-byte holding register is full.

10.3 FIFOs and errors

RX and TX FIFOs are 16 bytes deep. RX overrun: pushing into a full FIFO drops the new byte and latches the overrun sticky bit (UART_STATUS bit3). Framing errors latch bit2. Both sticky bits clear when the application reads UART_DATA. FIFO full/empty flags feed UART_STATUS bits 0/1. The bootloader depends on these semantics: it polls RX_VALID, drains promptly, and treats sticky error bits as a reason to abort the current frame (Section 12.5).

---

11. Mode selection and boot sequence

11.1 MODE pin semantics

MODE is a **boot strap**, not a run-time switch. It is sampled through a two-flop synchronizer during reset and latched when reset deasserts. Farad-1 note (Section 5, B6): there is no dedicated MODE pin — DIP0 (p84) plays the role, raw-high when open; `s8_top` therefore parameterizes `MODE_INVERT = 1` so that the logical polarity below (1 = programming) holds: switch closed → programming, switch open → run application (board default).

- Latched MODE = 1 → CPU held in reset; boot controller enters PROG (programming) state.
- Latched MODE = 0 → boot controller enters BOOT (application start) state.

During RUN, MODE transitions are ignored until the next hard reset. The `RUN` packet command and CPU_CONTROL.soft_reset perform a **warm hand-off**: soft reset re-enters the BOOT flow using the *latched* mode value, so a warm reset on a board strapped to MODE=1 returns to the bootloader, and MODE=0 returns to application validation. There is no way to enter PROG without a hard reset with MODE high — this is deliberate (it prevents a running application from being knocked into programming state by a serial glitch).

11.2 Boot flow

```
POWERON / hard reset
    │
    ▼
INIT: synchronize rst_n, sample+synchronize MODE (≥2 clk settle)
    │
    ├─ latched MODE=1 ──► PROG: cpu_reset held; packet loop:
    │                          HELLO ⇄, BEGIN(erase staging slot),
    │                          DATA×N (program), VERIFY (readback CRC),
    │                          COMMIT (metadata), RUN (release CPU)
    │
    └─ latched MODE=0 ──► BOOT:
                          read JEDEC ID (0x9F) → compare family (B8)
                          read both metadata records
                          validate (magic, version, slot match, marker,
                                    length ≤ 8192, CRC16 over image)
                          pick highest generation valid record
                          copy image from slot → program RAM
                          ▲ if no valid image: set BOOT_ERR, fall back to
                            PROG (a board with no image must still accept uploads)
                          release cpu_reset → RUN
```

RUN state: boot controller idles; CPU_CONTROL writes are honored (resume after HALT; soft reset re-enters BOOT). UART remains owned by `uart_regs`; the application polls it as the echo program does.

11.3 Copy details

The loader reads the chosen slot with op READ (0x03) in chunks of ≤ 512 bytes (two flash pages) into program RAM words, honoring the even-byte-high rule of Section 7.1. The copy is verified by the CRC computed during the metadata validation read (recompute during copy; mismatch → BOOT_ERR=2 and fall back to the other slot, then to PROG).

---

12. Packet protocol v1 (byte-exact)

All multi-byte fields are **big-endian**.

12.1 Frame layout

Offset	Field	Size	Definition
0	SYNC0	1	0x53 ('S')
1	SYNC1	1	0x38 ('8')
2	VERSION	1	0x01
3	COMMAND	1	See 12.3
4	SEQ	2	Monotonic host counter, starts at 1 per session (per BEGIN)
6	ADDR	4	Command-specific (see 12.3)
10	LEN	2	Payload length, 0..256
12	PAYLOAD	LEN	Command data
12+LEN	CRC16	2	CRC over bytes [2 .. 11+LEN] (VERSION through end of PAYLOAD)

Maximum frame size: 12 + 256 + 2 = 270 bytes.

12.2 CRC16

CRC16-CCITT-FALSE: polynomial 0x1021, initial value 0xFFFF, no input reflection, no output reflection, no final XOR, MSB-first byte processing, result transmitted high byte first. **Check value: CRC16("123456789") = 0x29B1.** Both the Rust crate and the RTL CRC unit must reproduce this check value in tests before any packet work starts.

12.3 Commands

CMD	Name	ADDR	LEN / PAYLOAD	Device action
0x01	HELLO	0	0	Session handshake; ACK payload: [proto_ver, max_payload_hi, max_payload_lo, features, mfr_id, dev_id] (6 bytes; mfr/dev from the last JEDEC read).
0x02	BEGIN	slot (0=A, 1=B)	4: image_len (2), image_crc16 (2)	Validates len (2..8192, even); erases the two sectors of the chosen slot; arms staging state.
0x03	DATA	image byte offset	1..256: raw bytes	Offset must be a multiple of 256 and offset+len ≤ image_len. Programs one flash page.
0x04	VERIFY	0	0	Reads back the staged image, computes CRC16, compares with BEGIN's image_crc16. ACK payload on success: [crc_hi, crc_lo].
0x05	COMMIT	0	0	Writes the metadata record for the staged slot (Section 13.5), marker byte last.
0x06	RUN	0	0	Warm hand-off (Section 11.1): releases CPU reset if an image is committed; otherwise NAK BAD_STATE.
0x07	ABORT	0	0	Discards staging state; no flash changes after this point.
0x08	STATUS	0	0	ACK payload: [state, last_error, gen_hi, gen_mid, gen_mid2, gen_lo] (6 bytes).

12.4 Responses

Every command receives exactly one response frame, using the same framing, with:

- COMMAND = request COMMAND | 0x80,
- SEQ echoed,
- ADDR echoed,
- LEN = 1 (+ command-specific data),
- PAYLOAD[0] = status code; any additional bytes follow (see the command table).

Status	Name	Meaning
0x00	OK	Command accepted and completed
0x01	BAD_CRC	Frame CRC mismatch (frame dropped silently; no ACK is sent — the host retries by timeout)
0x02	BAD_LEN	LEN invalid for the command or exceeds the negotiated maximum
0x03	BAD_SEQ	Sequence number out of order (not the duplicate case below)
0x04	BAD_OFFSET	DATA offset unaligned, out of range, or not the expected next offset
0x05	BAD_STATE	Command not valid in the current session state (e.g. DATA before BEGIN, COMMIT before VERIFY)
0x06	FLASH_ERR	SPI/flash operation failed (timeout, verify mismatch)
0x07	NO_IMAGE	RUN requested but no committed image exists
0xFF	UNKNOWN	Unrecognized command or internal decode failure

BAD_CRC frames produce **no response** (the device cannot trust the frame enough to attribute it); the host discovers this by ACK timeout.

12.5 Sequencing, duplicates, timeouts

- SEQ starts at 1 with each BEGIN and increments by 1 per host frame. The device accepts: `SEQ == expected` (normal) and, for DATA only, `SEQ == expected - 1` with identical ADDR and LEN (duplicate retransmission) → ACK OK, payload ignored. Any other SEQ → BAD_SEQ.
- DATA offsets must arrive in order: the device tracks `next_offset`; a frame whose ADDR equals `next_offset` advances it; a frame whose ADDR equals the previous offset is the duplicate case; anything else is BAD_OFFSET.
- Device inter-byte timeout: 250 ms between bytes of one frame → the partial frame is discarded, state unchanged, parser returns to SYNC hunt. Staging state (post-BEGIN) persists across frame aborts until ABORT or COMMIT or a new BEGIN.
- Host: ACK timeout 2 s, up to 3 retransmissions of the identical frame (safe because duplicates are idempotent). Host aborts the session on the third consecutive failure.

- Sticky RX error bits (framing/overrun) observed at any point cause the device to discard the partial frame and resynchronize.
- Ordering enforced by the device: HELLO may arrive in any state; DATA/VERIFY/COMMIT require staging; RUN requires a committed image or an already-valid boot image.

---

13. W25Q64 SPI flash subsystem

13.1 Commands used

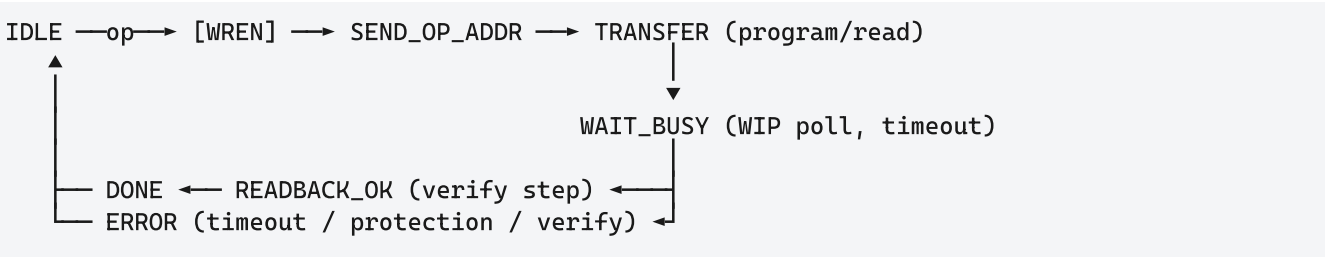
Command	Opcode	Protocol
Read JEDEC ID	0x9F	Send 0x9F, read 3 bytes (mfr, type, capacity). Expected for the Farad-1's W25Q64BV: **0xEF 0x40 0x17**; mismatch → BOOT_ERR=3.
Read Status Register-1	0x05	Send 0x05, read 1 byte while CS low; bit0 = WIP (busy), bit1 = WEL.
Write Enable	0x06	Single opcode; sets WEL. Required before program/erase.
Read Data	0x03	Opcode + 24-bit address + streamed read bytes (any length).
Page Program	0x02	WREN first; opcode + 24-bit address + 1..256 data bytes; bytes wrap inside the page if the host overruns — **the controller must never emit more than 256 bytes or cross a page boundary** (Section 9).
Sector Erase	0x20	WREN first; opcode + 24-bit sector address; then poll WIP with timeout.

Timing bounds used by the controller (generous superset of datasheet maxima): page program ≤ 3 ms, sector erase ≤ 400 ms, WIP poll timeout 1 s overall. WP#/HOLD# (if bonded) are tied or driven per B8; the controller assumes they are inactive.

13.2 Byte engine vs command controller

Two modules, as in the original design intent: `spi_byte_engine` does bit-level work only (mode 0: MOSI changes on falling SCLK edge, MISO sampled on rising edge, SCLK idle low, MSB first, programmable divider). `w25q64_controller` owns CS# for the entire command duration, sequences opcodes/addresses/data, performs WREN, polls status, enforces timeouts, and reports a 3-bit error code. The byte engine must never raise CS; the controller must never shift bits.

13.3 Controller FSM



Erase skips TRANSFER and goes straight to WAIT_BUSY. Program issues WREN, sends op+addr+data, then WAIT_BUSY, then (for loader-initiated programs) participates in the VERIFY_READBACK step of the boot controller.

13.4 Storage layout (Farad-1: SHARED configuration flash — relocated base)

On the Farad-1 the W25Q64BV **is** the FPGA configuration flash (Section 5, B7): the bitstream sits at 0x000000 (~338 KB for the LX9 — measure the generated .bit and confirm). S8 storage therefore uses a relocated, parameterized layout. The base `S8_FLASH_BASE` defaults to **0x060000** (sector 192 — ≈48 KiB above a typical LX9 bitstream end) and lives in one shared constant set (`flash_layout` in RTL, mirrored in the Rust `s8-image` crate); scattered absolute addresses are forbidden.

Range (default base)	Size	Use
0x000000–0x05FFFF	384 KiB	**FPGA configuration region — S8 must never erase or program**
0x060000–0x060FFF	4 KiB	Metadata record sector **A** (record at offset 0)
0x061000–0x061FFF	4 KiB	Metadata record sector **B** (record at offset 0)
0x062000–0x063FFF	8 KiB	Application slot A
0x064000–0x065FFF	8 KiB	Application slot B
0x066000–0x7FFFFF	rest	Reserved; untouched by v1

Rules: (1) before Phase 5 leaves simulation, confirm `BASE ≥ (bitstream bytes rounded up to a 4 KiB sector) + 1 sector` using the actual .bit file; if a future design grows the bitstream, raise BASE — never shrink it. (2) The bootloader, packet protocol (ADDR fields stay image-relative — unchanged), and host image packer all consume the base from the single constant set. (3) The iMPACT .mcs flow writes from address 0; never regenerate an .mcs with a shifted start address.

Rationale for separate metadata sectors: a commit erases only the *inactive* record's sector, so the other record survives a power cut at any point — the redundancy actually works. Slots are 8 KiB = program RAM capacity exactly (fixes original issue P2).

13.5 Metadata record (32 bytes, at offset 0 of a metadata sector)

Offset	Size	Field
0	4	Magic "S8MT" = 53 38 4D 54

Offset	Size	Field
4	1	Format version = 0x01
5	1	Slot id (0 = A, 1 = B)
6	4	Generation counter, u32 big-endian, monotonically increasing
10	2	Image length in bytes, u16 BE, 2..8192, even
12	2	Image CRC16 (CCITT-FALSE over the image bytes)
14	17	Reserved, must read 0xFF
31	1	Validity marker = 0xA5, **programmed last**

A record is valid iff: magic matches, version matches, slot id matches its sector's associated slot, marker == 0xA5, length in range, and the slot image's CRC16 equals the record's CRC. Invalid ⇒ treat the sector as empty (0xFF or garbage are both rejected by the magic check first).

13.6 Atomic update procedure (device side, on COMMIT)

1. Read back the full staged image; recompute CRC16; compare to the staged CRC from BEGIN. Mismatch → FLASH_ERR, staging kept (host may retry VERIFY/ABORT).
2. Choose the target record sector: the one whose record is *not* the current newest valid record (alternates naturally by generation parity).
3. Erase the target record sector (0x20 + timeout).
4. Program record bytes 0..30 (all but the marker); read back and compare.
5. Program the marker byte (offset 31); read back and confirm 0xA5.
6. Only now is the new image "committed". A power cut before step 5 leaves the old record intact and the new one invalid — boot falls back to the previous image (verified by the Phase-6 power-cut test).

The currently active slot is never erased: BEGIN for slot X requires that slot X is not the one the device booted from (NAK BAD_STATE otherwise). Alternating A/B across updates guarantees the previous image is still present when the new one is committed.

13.7 Boot-time validation (recap of the algorithm)

Read both records; filter to valid ones; pick the highest generation; verify its image CRC during (or after) the copy to program RAM; on copy-time mismatch, fall back to the other valid record; if none, BOOT_ERR=1 and fall back to PROG. Generation comparison is unsigned u32; wrap-around is accepted as a non-issue (centuries at human update rates).

---

14. Rust host toolchain

14.1 Workspace

`host/s8/` is a Cargo workspace: `s8-isa`, `s8-assembler`, `s8-image`, `s8-serial`, `s8-disasm`, `s8-cli`. Dependencies kept minimal: `clap` (CLI), `serialport` (host serial), `thiserror` (errors), and an in-repo CRC16

implementation in `s8-image` (do **not** pull a generic crc crate — the parameters are part of the protocol spec and must be visible in-tree, pinned by the 0x29B1 test).

14.2 `s8-isa`

The single source of truth for encodings.

```
pub enum Instruction { Ldi { rd: u8, imm: u8 }, Add { rd: u8, rs: u8 }, /* ...all 20-ish
        variants... */ }
pub fn encode(i: &Instruction) -> u16;
pub fn decode(w: u16) -> Result<Instruction, DecodeError>; // errors on any illegal
        pattern per 6.2
pub fn encode_str(s: &str) -> Result<u16, AsmError>;       // test helper: "LDI R0, 1" ->
        0x1001
```

Tests: the full golden-vector table of Section 6.7 (generated file included via `include_str!`), plus encode→decode→encode round trips over all legal encodings (exhaustive over the legal subspace), plus rejection tests for every illegal pattern class.

14.3 `s8-assembler`

Two-pass, per Section 6.8. Pass 1 assigns word addresses and collects labels/.equ constants; pass 2 parses operands, validates ranges, applies relaxation, encodes via `s8-isa`. Output: flat binary (big-endian word packing), optional Intel HEX, and an optional listing file (address, word, source line). Mandatory diagnostics with file:line:col. Tests: forward references, duplicate/undefined labels, range violations for each immediate format, branch relaxation both directions, `.org` overlap, `.byte` packing order, image-size limit (8192), and every example program in Section 6.9 assembling to its normative word list.

14.4 `s8-image`

`.s8img` container, 12-byte header + payload:

Offset	Size	Field
0	4	Magic “S8IM” = 53 38 49 4D
4	1	Format version = 0x01
5	1	Flags (bit0: payload is a raw binary; others reserved 0)
6	1	Preferred slot (0=A, 1=B, 0xFF=unspecified)
7	1	Reserved = 0
8	2	Payload length, u16 BE (2..8192, even)
10	2	Payload CRC16 (CCITT-FALSE)
12	N	Payload (program bytes, even-byte-high word packing)

`pack(bin, slot) -> S8img` / `unpack(s8img) -> (bin, slot)` with full validation. Tests: pack/unpack round trip, CRC corruption detection, length-limit rejection.

14.5 `s8-serial`

```rust
pub trait Transport { fn write(&mut self, buf: &[u8]) -> io::Result<()>; fn
        read_timeout(&mut self, buf: &mut [u8], ms: u32) -> io::Result<usize>; }
pub struct PortTransport { /* wraps serialport::SerialPort */ }
pub struct MockTransport { /* scripted request/response pairs for tests */ }
pub struct Client<T: Transport> { /* packet codec + session FSM */ }
impl<T: Transport> Client<T> {
    pub fn hello(&mut self) -> Result<Hello, ProtoError>;
    pub fn upload(&mut self, img: &S8img, slot: Slot, progress: impl FnMut(usize, usize))
        -> Result<(), ProtoError>;
    pub fn verify(&mut self) -> Result<u16, ProtoError>;
    pub fn commit(&mut self) -> Result<(), ProtoError>;
    pub fn run(&mut self) -> Result<(), ProtoError>;
    pub fn status(&mut self) -> Result<Status, ProtoError>;
}
```

Implements Section 12 exactly: CRC, big-endian fields, SEQ management, duplicate-idempotent DATA retries (3 × 2 s), BAD_* handling with distinct user-facing errors. The serial crate is isolated behind `Transport` so the protocol logic is fully unit-tested with `MockTransport` — no hardware, no timing flakiness. Tests: golden frame bytes for HELLO/BEGIN/DATA(1 page)/VERIFY/COMMIT/RUN (hand-computed vectors included in the spec repo), timeout/retry paths, BAD_CRC silent-drop path, duplicate-DATA idempotency, BAD_OFFSET on a skipped page.

14.6 `s8-disasm`

`decode`-driven; annotates labels when given the symbol map; used by `s8 disasm` and in round-trip tests (assemble → disassemble → reassemble → identical bytes) over all example programs.

14.7 `s8-cli`

```
s8 asm examples/gpio.s8 -o build/gpio.bin [--listing build/gpio.lst] [--hex]
s8 disasm build/gpio.bin
s8 image build/gpio.bin --slot A -o build/gpio.s8img
s8 upload build/gpio.s8img --port COM5 --baud 115200      # port auto-scan allowed if --port
s8 verify --port COM5
s8 status --port COM5
s8 monitor --port COM5 --baud 115200                      # raw terminal, local echo off, he
```

Port names: never hard-code COM5 or /dev/ttyUSB0; enumerate candidates with `serialport::available_ports()` and require confirmation when ambiguous. `cargo test` must pass with no hardware attached (all serial tests use MockTransport).

---

15. Xilinx ISE workflow

ISE 14.7 is the final release supporting Spartan-6; Vivado does not. Practical notes: the Windows installer ships both 32-bit and 64-bit variants (use the 32-bit toolchain on modern Windows if the 64-bit GUI misbehaves, or run in a Windows 7/10 VM or under Linux Wine); iSIM and the command-line flows (`xst`, `ngdbuild`, `map`, `par`, `trce`, `bitgen`) are scriptable and preferred for CI-like reproducibility over the GUI.

Sequence:

1. Create project with the **exact** full part string (B1) — including speed grade and package.
2. Add RTL to the synthesis source set; testbenches only to the simulation source set.
3. Set `s8_top` as synthesis top; run behavioral simulation of every testbench in Section 16.1 **before** synthesis.
4. Synthesis: verify inferred BRAM for `s8_program_ram` (must be BRAM, not distributed) and check utilization reports against the budget (Section 16.5).
5. Constraints: start from the Farad-1 UCF below — every LOC verified against the schematic renders (Section 5.2). The vendor example UCF (`BoardFiles/Farad1_UCF.ucf`) is correct on its LOCs (including `Serial_Tx = p41` / `Serial_Rx = p40`) but incomplete: it has **no IOSTANDARD anywhere, no PERIOD constraint, and none of the flash pins**. All three gaps are fixed here:

```
# S8 v1 constraints — Farad-1 (xc6slx9-2tqg144), verified against schematic rev 2016-12-29
NET "clk"        LOC = "P85" | IOSTANDARD = LVCMOS33;  TNM_NET = sys_clk;
TIMESPEC "TS_sys_clk" = PERIOD "sys_clk" 20.000 ns HIGH 50%;   # 50.000 MHz oscillator
NET "rst_n"      LOC = "P87" | IOSTANDARD = LVCMOS33;  # nFP_RST button, active low, RC-debo
NET "mode"       LOC = "P84" | IOSTANDARD = LVCMOS33;  # DIP0; open=1, closed=0; MODE_INVERT
NET "gpio<0>"    LOC = "P105" | IOSTANDARD = LVCMOS33; # LED0, active high
NET "gpio<1>"    LOC = "P104" | IOSTANDARD = LVCMOS33; # LED1, active high
NET "gpio<2>"    LOC = "P102" | IOSTANDARD = LVCMOS33; # LED2, active high
NET "gpio<3>"    LOC = "P101" | IOSTANDARD = LVCMOS33; # LED3, active high
NET "uart_rxd"   LOC = "P83" | IOSTANDARD = LVCMOS33;  # FT2232H Ch.B TXD (after FT_PROG ->
NET "uart_txd"   LOC = "P82" | IOSTANDARD = LVCMOS33;  # FT2232H Ch.B RXD
# DB9/MAX232 fallback: uart_rxd = P40, uart_txd = P41 (swap LOCs only)
NET "spi_sck"    LOC = "P70" | IOSTANDARD = LVCMOS33;  # flash CK (CCLK pin, net FP_PR_CLK)
NET "spi_cs_n"   LOC = "P38" | IOSTANDARD = LVCMOS33;  # flash CS# (CSO_B; board pull-up R6)
NET "spi_mosi"   LOC = "P64" | IOSTANDARD = LVCMOS33;  # flash DI (net MISO0)
NET "spi_miso"   LOC = "P65" | IOSTANDARD = LVCMOS33;  # flash DO (net MISO1)
NET "spi_wp_n"   LOC = "P62" | IOSTANDARD = LVCMOS33;  # flash WP# - drive high, no board pu
NET "spi_hold_n" LOC = "P61" | IOSTANDARD = LVCMOS33;  # flash HOLD# - drive high, no board
```

6. Implementation + timing: all paths must meet the PERIOD constraint with margin; investigate every timing summary line before generating a bitstream.
7. Bitstream via iMPACT/JTAG; keep a known-good recovery bitstream in `build/` with its hash recorded.
8. Reproducibility record per bitstream: ISE version, git revision, UCF content, timing summary, utilization summary, bitstream SHA-256.

UG380 (Spartan-6 Configuration User Guide) is required reading: the Farad-1's W25Q64BV is the configuration flash (Section 5, B7) and S8 accesses it as ordinary user I/O after configuration completes — CCLK/CSO_B/MISO0-style pins become fabric-driven SPI signals per the UCF above.

---

16. Verification plan

16.1 RTL testbenches

Testbench	Covers	Key checks
`tb_s8_alu`	`s8_alu`	Every operation class against a behavioral model; exhaustive operand sweep for ADD/SUB (all 65536 pairs) verifying flags per Section 6.5 formulas; directed cases for each O/C corner (0x7F+1, 0x80−1, 0xFF+1, equal operands, etc.).
`tb_s8_regfile`	`s8_regfile`	Write/read ports, sync-write timing, no read-during-write hazard.
`tb_s8_cpu_programs`	CPU + program RAM	Loads normative programs from `tb/vectors/` (generated from `s8-isa`): GPIO words execute and drive the port value; UART echo program run against a UART BFM that injects bytes and checks echoes; golden vector sweep: one instruction per encoding, architectural state checked after each; HALT/resume; TRAP on opcode 0xE and on non-zero reserved bits; stack fault TRAP.
`tb_uart_loopback`	rx + tx + regs + baud_gen	Tie txd→rxd; stream 256 random bytes with random gaps; check data integrity, FIFO full behavior, sticky overrun/framing bits, framing error on a corrupted stop bit, tolerance to ±2% injected baud offset.
`tb_spi_byte_engine`	`spi_byte_engine`	Mode-0 timing against a bit-accurate reference; MOSI/MISO ordering; divider correctness for DIV ∈ {2,4,8}.
`tb_w25q64_model`	behavioral flash model	The model itself: JEDEC ID read, WEL/WIP behavior, page wrap protection, sector erase, busy timing, and its own fault-injection hooks.
`tb_boot_packets`	packet parser + CRC + session FSM	Golden CRC (0x29B1 check vector first); golden frame bytes from Section 14.5's test vectors; fuzz/negative: bad sync, bad CRC, bad LEN, bad SEQ, duplicate DATA (idempotent), skipped page (BAD_OFFSET), commands out of order (BAD_STATE), truncated frames (timeout recovery), oversize LEN.

Testbench	Covers	Key checks
`tb_full_boot`	entire boot path	MODE=1: full upload session against `tb_w25q64_model`, COMMIT, RUN, then MODE=0 cold boot: image copied to program RAM, CPU executes it and emits the expected UART bytes; power-cut injection at each COMMIT step → previous image still boots; slot-selection: newer generation wins; corrupted image → fallback.

16.2 Behavioral W25Q64 model (`tb_w25q64_model.v`)

A Verilog model with an 8 MiB array (or 64 KiB sparse window — the layout only needs 0x0000–0x5FFF plus wrap tests — choose sparse to keep sim fast, but keep addresses honest). Behaviors: JEDEC ID bytes (parameterized, default a Winbond ID), WEL/WIP status semantics, page-program bit-clear-only semantics (programming over non-errored bits must be modeled as the real device behaves: AND semantics), sector erase timing (parameterized fast for sim), busy delay parameters, and fault hooks (drop N CS edges, delay busy N cycles, corrupt byte at address X) driven by plusargs.

16.3 Host-side tests

All `cargo test` suites run offline: golden vectors, assembler diagnostics, image pack/unpack, protocol sessions over MockTransport, retry/timeout logic with simulated clock via injectable sleep. A `--locked` build must be reproducible.

16.4 Hardware acceptance tests (per phase)

Phase	Test	Pass
2	GPIO program on board	LED0 (p105) lights and holds the programmed level: `LDI R0,1` + `OUT GPIO_OUT,R0` turns it on (active high); scope confirms static level after the loop
3	UART loopback	Board echoes PC bytes at 115200 for ≥ 10 min with zero losses at the offered rate
3	Framing stress	Host sends 0x00, 0xFF, 0x55, 0xAA patterns; echo integrity maintained
5	JEDEC ID	`s8 status` (or UART console command in the boot controller) reports the expected family ID
6	Full upload	`s8 upload` + `s8 verify` + power-cycle with MODE=0 → application runs from flash

Phase	Test	Pass
6	Power-cut sweep	Cut power during COMMIT at ≥ 5 randomized delays; board always boots either the old or the new image, never a bricked state
6	Slot alternation	Two successive uploads land in A then B; generation increments monotonically

16.5 Budgets and traceability

Budgets (XC6SLX9: 1,430 slices, 32 BRAM18 blocks, 90 Kb distributed RAM — Section 5.4): program RAM = 4 BRAM18; UART FIFOs + data RAM = distributed RAM (0 BRAM); total slice utilization target ≤ 60 % (~860 slices) at the end of Phase 6 (margin for the compiler/debugger phases).

Every numbered requirement in this document maps to at least one testbench or acceptance test; the mapping table is maintained in `docs/verification-plan.md` with columns (requirement tag, section, test ID, status). Requirement tags are assigned per section (e.g. S8-12.5-03 = Section 12.5, third bullet).

---

17. Safety, reliability, and failure handling

Consolidated hard rules (each is testable; several are already in the phase gates):

1. Pin assignments come from the schematic only; every top-level port gets LOC + IOSTANDARD; no placeholder survives into implementation.
2. 3.3 V logic levels only on user I/O; never RS-232 voltages; never 5 V unless the bank spec explicitly allows it.
3. CPU and bootloader ownership are mutually exclusive: CPU reset is held for the entire PROG state.
4. Every length, address, loop, and wait is bounded; the assembler rejects images > 8192 bytes; the device rejects LEN > 256 and offsets ≥ image_len.
5. The active slot is never erased; the metadata marker is written last; both metadata records live in separate erase sectors.
6. Flash BUSY is always polled with a timeout; WEL is checked; programmed data is read back.
7. The FPGA configuration region is untouchable — the flash IS shared on the Farad-1 (B7); layout constants live in one place (`S8_FLASH_BASE`, Section 13.4).
8. JTAG reconfiguration with a known-good recovery bitstream is always possible (never lock configuration pins as user I/O in the constraints).
9. All asynchronous inputs pass two-flop synchronizers (MODE, rst_n, uart_rxd, miso sampled by the divider — synchronous to sck by construction).
10. CRC detects corruption; it is not authentication. Document this wherever uploads are described.
11. Farad-1 flash pins WP# (p62) and HOLD# (p61) have **no board pull-ups**: the boot controller drives both HIGH from configuration release onward, before any flash command. A floating HOLD# can stall the SPI bus.
12. The DB9/MAX232 path is electrically safe as designed (5 V translator, but the FPGA-side RX passes through a 5.6 K/10 K divider ≈ 3.2 V max, input-only). Never bypass the divider; never treat the DB9 pins as logic-level.

18. Phased execution plan

Each phase has a deliverable and a hard exit gate. Do not start the next phase until the gate is demonstrated, ideally by an automated test.

Phase	Deliverable	Exit gate
0. Foundation	This spec frozen in `docs/`; board facts B1–B9 verified or explicitly deferred; vector generator running	Golden vectors reproduce in both Node generator and Rust tests; UCF skeleton exists with placeholders still unfilled
1. CPU core	`s8_alu`, `s8_regfile`, `s8_cpu`, `s8_program_ram`, `s8_data_ram`, `s8_bus` + testbenches	`tb_s8_alu`, `tb_s8_regfile`, `tb_s8_cpu_programs` pass; golden vector sweep green; HALT/TRAP/stack-fault cases green
2. First hardware	`s8_top` minimal (CPU + GPIO), Farad-1 UCF from Section 15, ISE project runs clean	GPIO demo (Section 6.9.1) observed on LED0 (p105, active high); timing closure with margin; utilization recorded
3. UART	`baud_gen`, `uart_rx`, `uart_tx`, `uart_regs`, echo program	`tb_uart_loopback` green; on-board echo survives the 10-minute stress test
4. Boot protocol	`s8_boot_ctrl` packet parser + session FSM; host `s8-serial` + `s8-cli` upload/verify commands	`tb_boot_packets` green (including fuzz/negative); a full host↔RTL upload session runs against the flash model in simulation
5. SPI discovery	`spi_byte_engine`, `w25q64_controller`, JEDEC read path wired into the boot controller	`tb_spi_byte_engine` and model tests green; on-board JEDEC read returns **0xEF 0x40 0x17** (if not: continuity check + p64/p65 swap per Section 5.3)
6. Persistent slots	Full loader, metadata commit, A/B selection, `s8 image` / `s8 upload` end-to-end	`tb_full_boot` green incl. power-cut injection; hardware acceptance table (16.4) passes; two alternating uploads verified on hardware
7. Compiler	C-like frontend per Section 19	Compiled programs match assembler-level equivalents in simulation; ABI conformance tests pass
8. Debugger	`s8 disasm` symbols, monitor improvements, status/break support	Repeatable debug workflow demonstrated: upload → run → observe → diagnose from symbols

Phase ordering rationale: simulation-heavy phases (1, 4) proceed without board facts; hardware risk is retired early (Phase 2 proves the toolchain, constraints, and board interface before any protocol work); flash risk lands late, once the recovery path (JTAG + known-good bitstream) is already proven.

19. Future work (specified, not scheduled)

19.1 Compiler (Phase 7)

Pipeline: source → lexer → parser/AST → semantic analysis (u8, bool, void; no dynamic allocation) → IR → optional constant folding / DCE → instruction selection (S8 ISA v1) → register allocation → branch relaxation → `s8-isa` encoding → `.s8img` → existing uploader. Statements: declarations, assignment, if/else, while, functions, return. Built-ins only when backed by real runtime/hardware: `gpio_read`, `gpio_write`, `uart_available`, `uart_read`, `uart_write`, `delay`. Debug info: JSON symbol map (source line ↔ word address) feeding `s8 disasm`.

19.2 Instruction-fetch from external flash

If images must exceed 8 KiB: an XIP fetch interface over the SPI controller with a simple direct-mapped instruction cache is the specified direction. Explicitly deferred; BRAM copy remains the v1 mechanism.

19.3 ABI proposal (freeze when the compiler starts)

R0 = return value and scratch; R1–R2 = arguments; R3–R5 = caller-saved temporaries; R6–R7 = callee-saved; SP grows down from 0xEF; 16-entry hardware stack implies a software spill strategy for deep call chains (compiler must either prove depth ≤ 16 or spill to 0x00–0xDF); prologue = PUSH callee-saved, epilogue = POP in reverse.

---

20. References

- UG380 — Spartan-6 FPGA Configuration User Guide. Configuration modes, SPI flash wiring, user-mode access rules. Required: the Farad-1's W25Q64BV IS the configuration flash (B7); S8 accesses it as user I/O after configuration. https://docs.amd.com/v/u/en-US/ug380
- UG382 — Spartan-6 FPGA Memory Resources User Guide. BRAM primitives, port configurations, inference rules for `s8_program_ram`/`s8_data_ram`.
- UG612 — ISE Synthesis and Simulation Design Guide; UG621 — ISim simulation guide (command-line flows preferred).
- Xilinx ISE Design Suite 14.7 (final Spartan-6 release): https://www.xilinx.com/products/design-tools/ise-design-suite.html
- Spartan-6 package pinout files: https://www.xilinx.com/support/package-pinout-files/spartan-6-pkgs.html (board routing and bank voltages still require the schematic).
- Winbond W25Q64 family datasheet (use the exact suffix): https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?__locale=en
- CRC16-CCITT-FALSE parameters and check value 0x29B1 (canonical catalogue value; reproduced in-tree by `s8-image`'s CRC unit test).
- Farad-1 board documentation (in-repo): `BoardFiles/Farad-1-Schematic.pdf` (6 sheets, rev 2016-12-29), `BoardFiles/Farad-1-User-Manual.pdf`, `BoardFiles/Farad1_UCF.ucf` (vendor example: LOCs correct, missing IOSTANDARD/PERIOD constraints — build from Section 15 instead).

Appendix A — Guide for an implementing agent

1. Start at Phase 0/1: create the repo tree (Section 3), copy the frozen tables (Sections 6, 12, 13.5) into `docs/`, and run `tools/gen_isa_vectors.js` to populate `tb/vectors/` and the Rust golden-vector include file.
2. Build `s8-isa` first and make its tests pass; everything else depends on it.
3. Build `s8_alu` + `tb_s8_alu` next; the exhaustive flag sweep catches formula misunderstandings before they spread into the CPU.
4. When writing the CPU FSM, keep the state list of Section 8.2 verbatim; add states only with a written justification in the module header.
5. The two most bug-prone seams in the whole design are (a) the byte order between image bytes and 16-bit words (even byte = high byte — one helper function, used everywhere, no ad-hoc swapping) and (b) the DATA packet offset/page-boundary rules (256-aligned offsets, device rejects everything else). Test both seams directly, not just through the happy path.
6. Never "fix" a failing golden vector by editing the vector. The vector is the spec; the code is suspect.
7. Board facts for the Farad-1 are already verified and recorded in Section 5 (evidence: schematic page renders inspected visually, page by page). On first bring-up, perform the Section 5.3 checks — FT_PROG EEPROM backup then Channel B → UART; flash-pin JEDEC confirmation; record the results in `docs/board-bringup.md`. The vendor UCF ships without IOSTANDARD/PERIOD constraints: always build from the Section 15 UCF.

End of specification.